In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, roc_curve, auc,
    classification_report
)

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
```

In [3]:
```
df = pd.read_csv(r'C:\Users\User\Downloads\WA_Fn-UseC_-Telco-Customer-Churn.csv')

df.head()
df.info()
df.describe(include="all")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Out[3]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneServi
count	7043	7043	7043.000000	7043	7043	7043.000000	704
unique	7043	2	NaN	2	2	NaN	
top	7590-VHVEG	Male	NaN	No	No	NaN	Y
freq	1	3555	NaN	3641	4933	NaN	636
mean	NaN	NaN	0.162147	NaN	NaN	32.371149	Na
std	NaN	NaN	0.368612	NaN	NaN	24.559481	Na
min	NaN	NaN	0.000000	NaN	NaN	0.000000	Na
25%	NaN	NaN	0.000000	NaN	NaN	9.000000	Na
50%	NaN	NaN	0.000000	NaN	NaN	29.000000	Na
75%	NaN	NaN	0.000000	NaN	NaN	55.000000	Na
max	NaN	NaN	1.000000	NaN	NaN	72.000000	Na

11 rows × 21 columns

In [5]:
```
# TotalCharges sometimes comes as object - convert to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# Handle missing values (after conversion)
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

# Drop customerID - not useful as a feature
df = df.drop(columns=['customerID'])
```

In [7]:
```
# Encode target
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# Separate X, y
X = df.drop('Churn', axis=1)
y = df['Churn']

# One-hot encode categorical variables
X = pd.get_dummies(X, drop_first=True)
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [11]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [13]:
```
def evaluate_model(model, X_train, y_train, X_test, y_test, model_name="Model"):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") e

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print(f"=== {model_name} ===")
    print("Accuracy :", acc)
    print("Precision:", prec)
    print("Recall   :", rec)
    print("F1-score :", f1)
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    print("\nConfusion Matrix:\n", cm)

    # ROC + AUC
    if y_prob is not None:
        fpr, tpr, thresholds = roc_curve(y_test, y_prob)
        roc_auc = auc(fpr, tpr)

        plt.figure()
        plt.plot(fpr, tpr, label=f'{model_name} (AUC = {roc_auc:.3f})')
        plt.plot([0, 1], [0, 1], linestyle='--')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC Curve - {model_name}')
        plt.legend()
        plt.grid(True)
        plt.show()
    else:
        roc_auc = None

    return {
        "model": model_name,
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
        "auc": roc_auc
    }
```

In [15]:
```
results = []

# 1. Logistic Regression
lr = LogisticRegression(max_iter=1000)
results.append(evaluate_model(lr, X_train_scaled, y_train, X_test_scaled, y_test, "

# 2. K-Nearest Neighbors
```

```
knn = KNeighborsClassifier(n_neighbors=5)
results.append(evaluate_model(knn, X_train_scaled, y_train, X_test_scaled, y_test,

# 3. Decision Tree
dt = DecisionTreeClassifier(random_state=42)
results.append(evaluate_model(dt, X_train, y_train, X_test, y_test, "Decision Tree"

# 4. Random Forest
rf = RandomForestClassifier(n_estimators=200, random_state=42)
results.append(evaluate_model(rf, X_train, y_train, X_test, y_test, "Random Forest"

# 5. Neural Network (optional but recommended)
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=300, random_state=42)
results.append(evaluate_model(mlp, X_train_scaled, y_train, X_test_scaled, y_test,
```

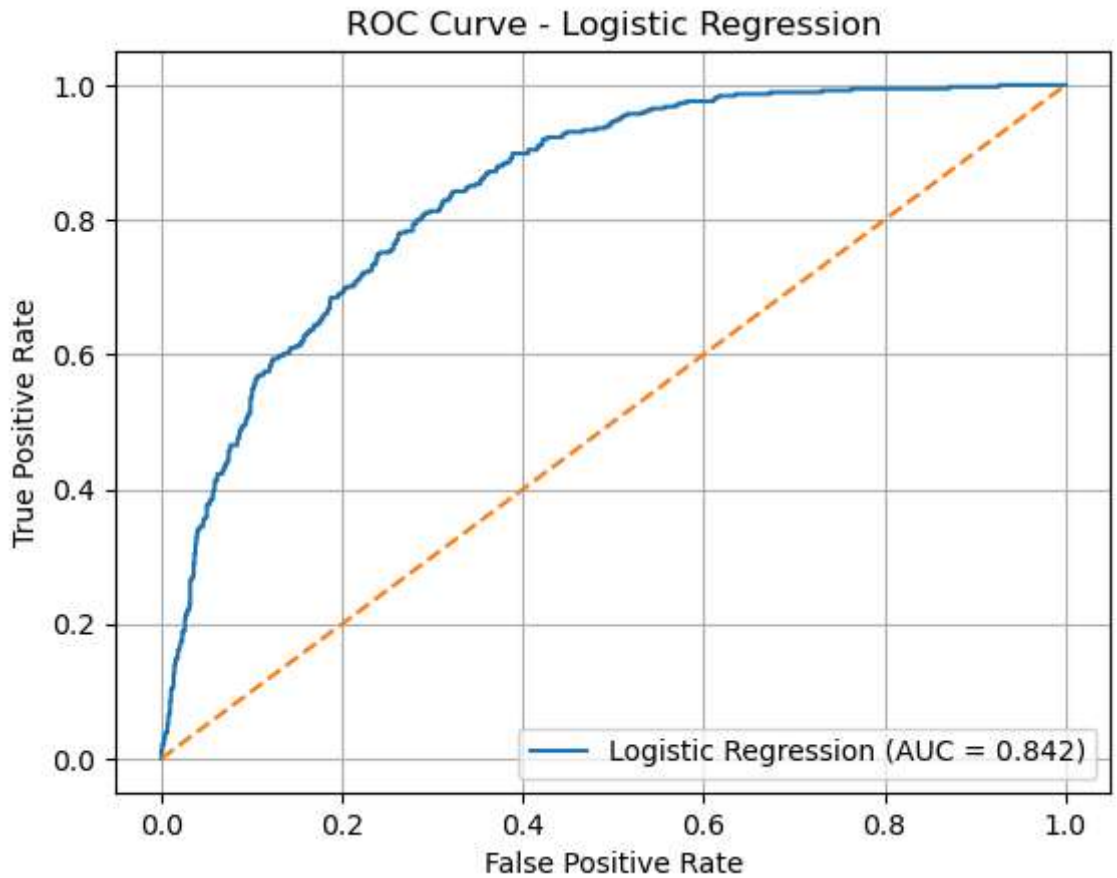
```
=== Logistic Regression ===
Accuracy : 0.8069552874378992
Precision: 0.6583850931677019
Recall   : 0.5668449197860963
F1-score : 0.6091954022988506

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.89      0.87      1035
           1       0.66      0.57      0.61       374

    accuracy                           0.81      1409
   macro avg       0.75      0.73      0.74      1409
weighted avg       0.80      0.81      0.80      1409


Confusion Matrix:
 [[925 110]
 [162 212]]
```



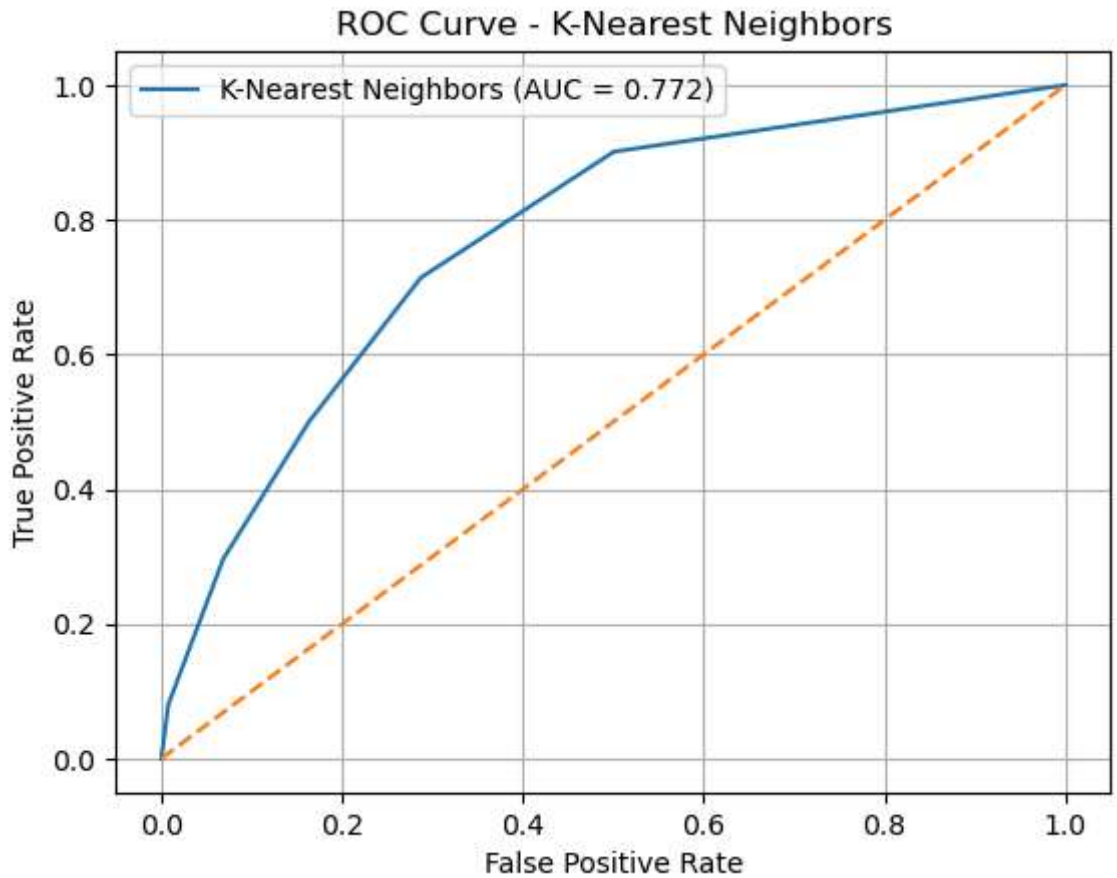

```
=== K-Nearest Neighbors ===
Accuracy : 0.7473385379701917
Precision: 0.5252808988764045
Recall   : 0.5
F1-score : 0.5123287671232877

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.84      0.83      1035
           1       0.53      0.50      0.51       374

    accuracy                           0.75      1409
   macro avg       0.67      0.67      0.67      1409
weighted avg       0.74      0.75      0.75      1409


Confusion Matrix:
 [[866 169]
 [187 187]]
```



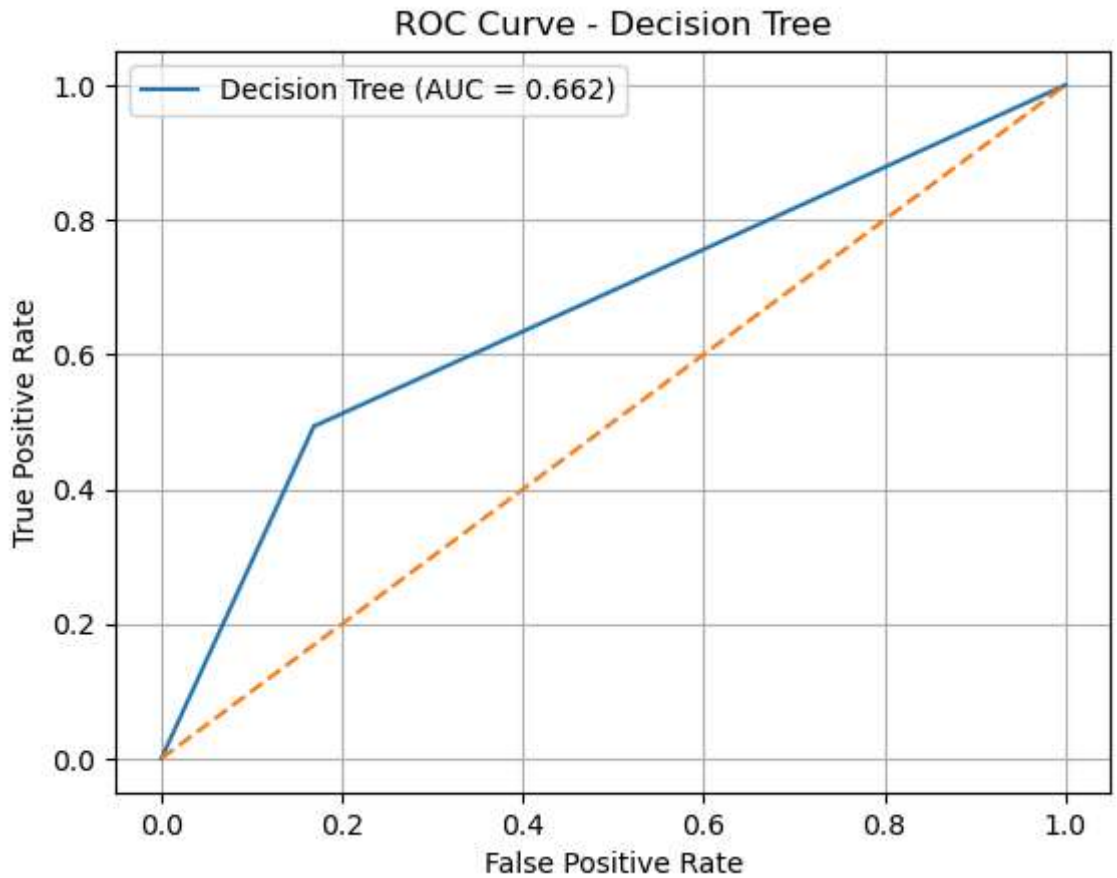

```
=== Decision Tree ===
Accuracy : 0.7416607523066004
Precision: 0.5138888888888888
Recall   : 0.4946524064171123
F1-score : 0.5040871934604905

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.83      0.83      1035
           1       0.51      0.49      0.50       374

    accuracy                           0.74      1409
   macro avg       0.67      0.66      0.66      1409
weighted avg       0.74      0.74      0.74      1409


Confusion Matrix:
 [[860 175]
 [189 185]]
```



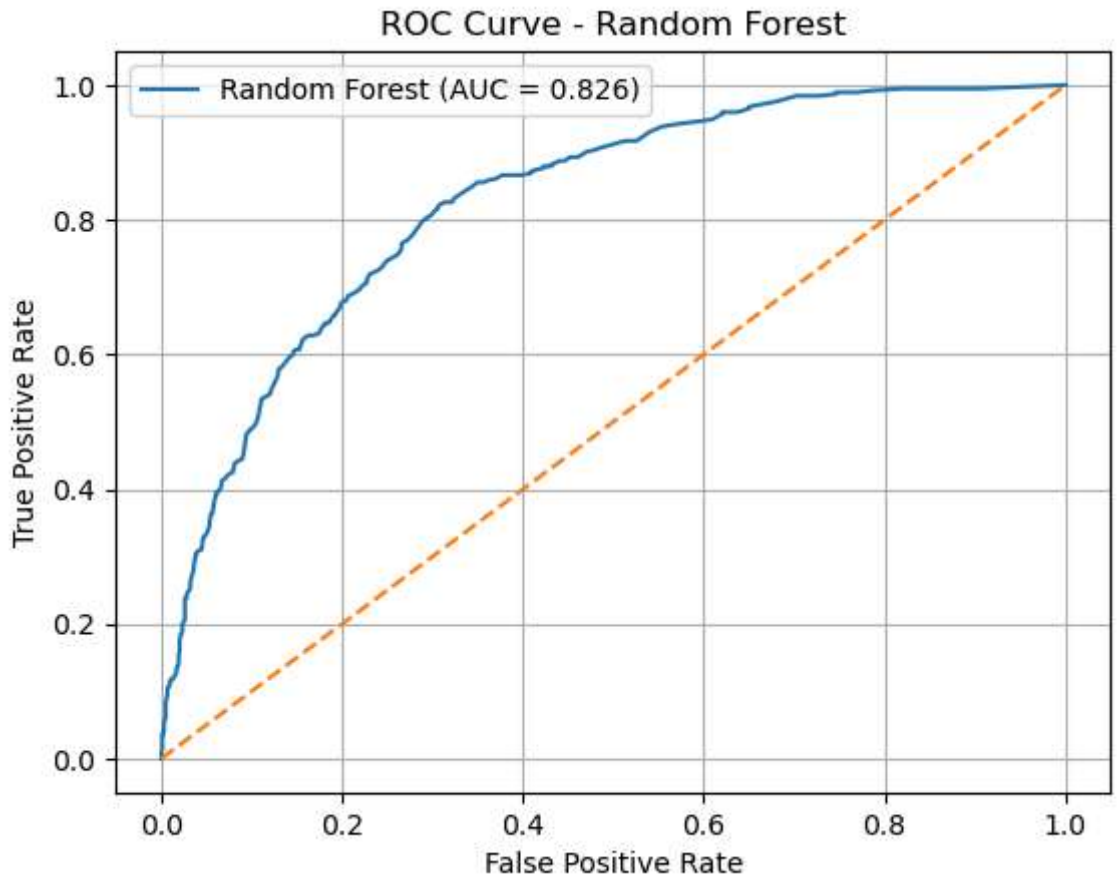

```
=== Random Forest ===
Accuracy : 0.7899219304471257
Precision: 0.6335616438356164
Recall   : 0.4946524064171123
F1-score : 0.5555555555555556

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.90      0.86      1035
           1       0.63      0.49      0.56       374

    accuracy                           0.79      1409
   macro avg       0.73      0.70      0.71      1409
weighted avg       0.78      0.79      0.78      1409


Confusion Matrix:
 [[928 107]
 [189 185]]
```



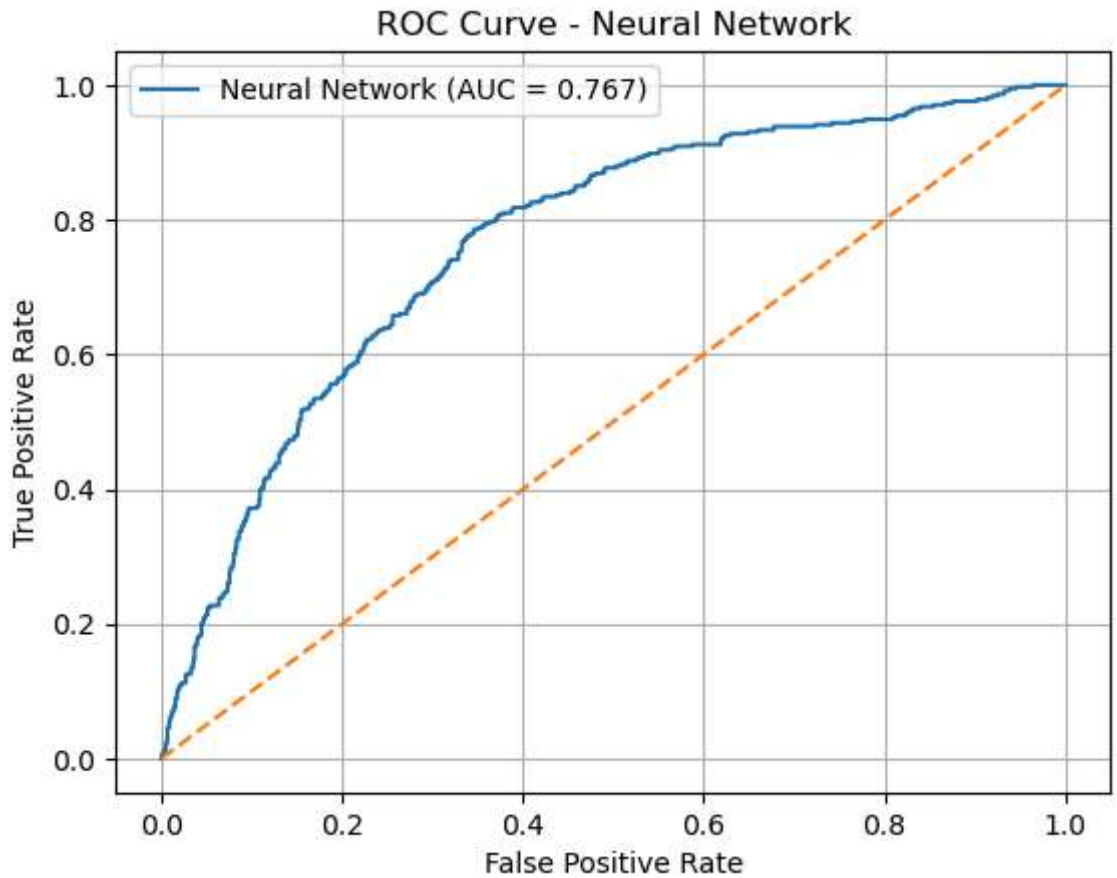

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_percept
ron.py:690: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reach
ed and the optimization hasn't converged yet.
  warnings.warn(
=== Neural Network ===
Accuracy : 0.752306600425834
Precision: 0.5335120643431636
Recall   : 0.5320855614973262
F1-score : 0.5327978580990629

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.83      0.83      1035
           1       0.53      0.53      0.53       374

    accuracy                           0.75      1409
   macro avg       0.68      0.68      0.68      1409
weighted avg       0.75      0.75      0.75      1409


Confusion Matrix:
 [[861 174]
 [175 199]]
```



In [17]:
```
results_df = pd.DataFrame(results)
results_df
```

Out[17]:

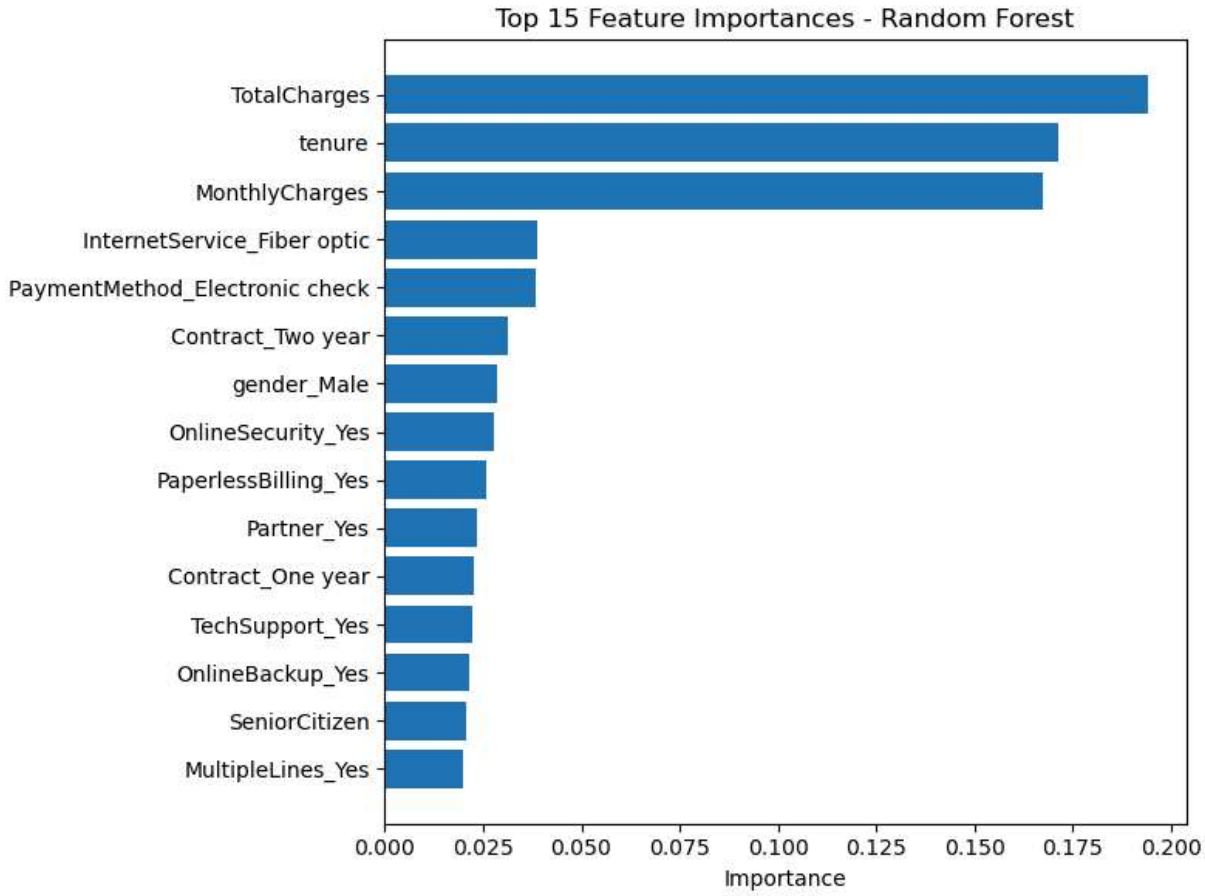
	model	accuracy	precision	recall	f1	auc
0	Logistic Regression	0.806955	0.658385	0.566845	0.609195	0.841585
1	K-Nearest Neighbors	0.747339	0.525281	0.500000	0.512329	0.771797
2	Decision Tree	0.741661	0.513889	0.494652	0.504087	0.662297
3	Random Forest	0.789922	0.633562	0.494652	0.555556	0.826493
4	Neural Network	0.752307	0.533512	0.532086	0.532798	0.767243

In [19]:
```
importances = rf.feature_importances_
feature_names = X.columns

feat_imp = pd.DataFrame({
    "feature": feature_names,
    "importance": importances
}).sort_values(by="importance", ascending=False)

plt.figure(figsize=(8, 6))
plt.barh(feat_imp['feature'][:15], feat_imp['importance'][:15])
plt.gca().invert_yaxis()
plt.xlabel("Importance")
plt.title("Top 15 Feature Importances - Random Forest")
```

```
plt.tight_layout()
plt.show()
```



In [ ]: